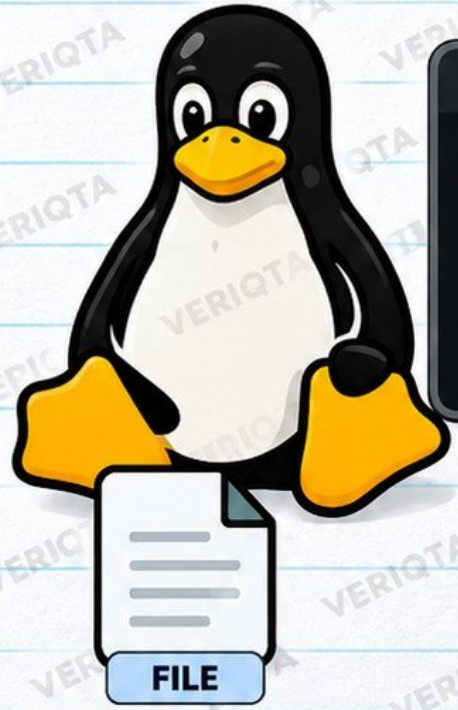
VERIQTA JUNIOR ENGINEER

Linux File Permissions: The Beginner's Visual Guide



```
$ ls -l
-rwxr-xr--  1 user  dev   1024 script.sh
drwxr-xr-x  2 user  dev   4096 project/
-rw-------  1 user  user   260 .env
-rwxr-xr-x  1 user  dev   2048 app
```




Permission	Meaning
r	Read
w	Write
x	Execute

-rw-r--r--
Owner Group Others

drwxr-xr-x
Owner Group Others

Understand how permissions work

Use commands with confidence

Troubleshoot real permission errors

Apply best practices inreal servers

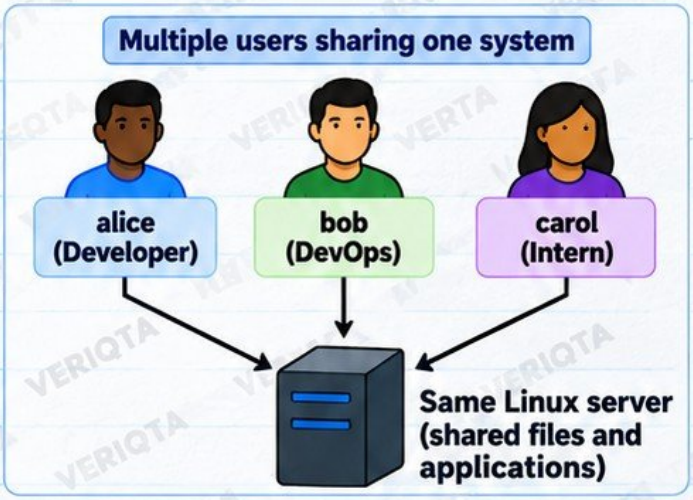
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Why Linux File Permissions Exist

1. Linux is a multi-user system

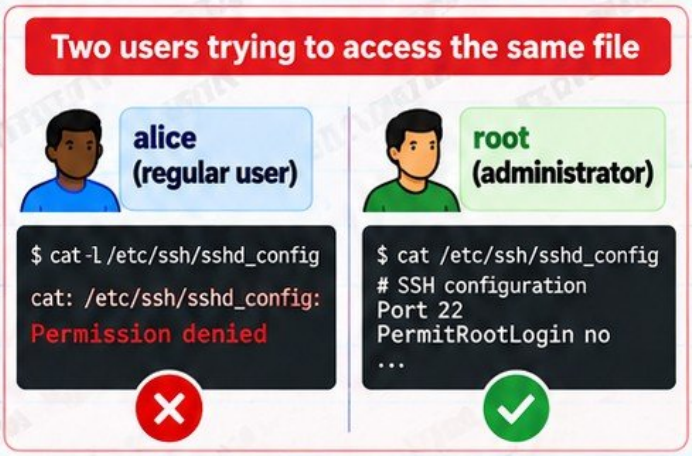
- Many users, applications, and services share the same server.
- Each user should only access the files and directories they are allowed to use.
- File permissions help Linux control who can read, write, or execute a file or directory.



2. Example: A protected configuration file

The /etc/ssh/sshd_config file controls the SSH server. Only the root user should be able to modify it.

```
$ ls -l /etc/ssh/sshd_config
-rw------- 1 root root  3220 Jul 10 08:22
```



3. What could happen without permissions?

1. Accidental changes

A user or application could modify important system files by mistake.

2. Malicious changes

An attacker could modify configurations to gain full control of the system.

3. System instability

Critical files could be deleted or corrupted, causing services to stop working.

4. Data exposure

Sensitive files such as keys, passwords, and user data could be read by anyone.

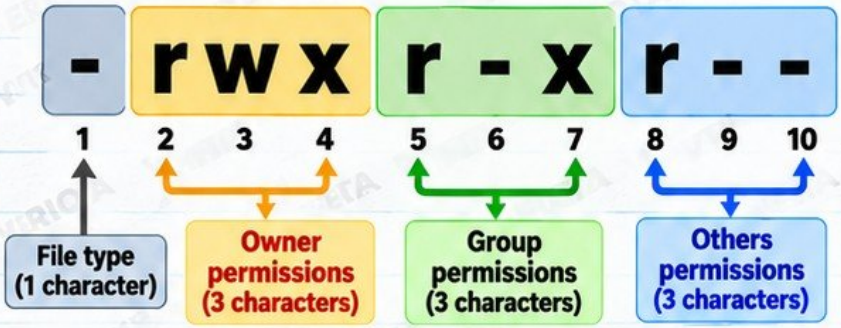
Engineering insight:

A permission error is often a security boundary working correctly.
It means Linux is doing its job by blocking an action that the current user is not allowed to perform.

Decode -rwxr-xr-- Without Memorizing

1. The 10 characters from ls -l

When you run **ls -l**, the first 10 characters show the file type and permissions.



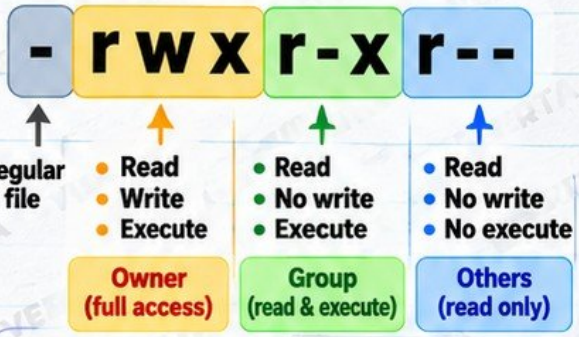
2. File type (character 1)

-	Regular file
d	Directory
l	Symbolic link
c	Character device
b	Block device
p	Named pipe
s	Socket

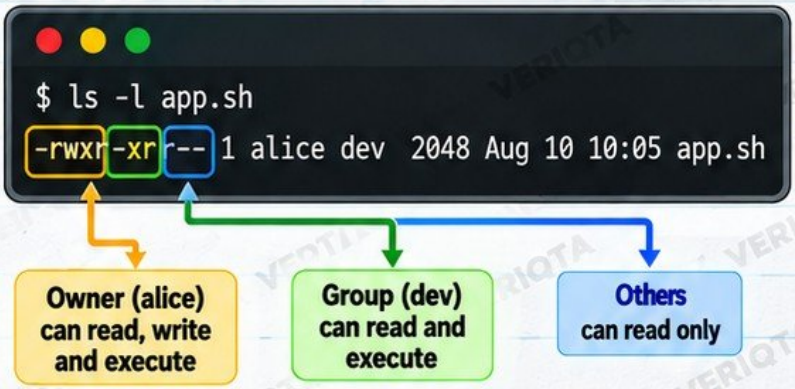
3. Permission characters (characters 2–10)

Symbol	Meaning	What it allows
r	Read	View the file content (or list directory names)
w	Write	Modify the file content (or create/delete files in a directory)
x	Execute	Run the file as a program (or enter/traverse a directory)
-	No permission	This permission is not allowed

4. Break down the example



5. Real example from a Linux server



6. Practice: Try it yourself

1. Create a file and check its permissions.
```
touch test.txt
ls -l test.txt
```
2. Identify the file type, owner, group, and permission for each group.
3. Create a directory and check its permissions.
```
mkdir mydir
ls -ld mydir
```
4. Change the permissions and verify the result.
```
chmod 640 test.txt
ls -l test.txt
```

Remember: The 10 characters show the file type and who can read, write, and execute. You do not need to memorize them. Just break them into parts and understand what each part means.

Owner, Group, and Others: Who Gets Access?

1. How Linux decides who gets access

1. Linux checks the file owner first.
   If you are the owner, your permissions are used.
2. If you are not the owner, Linux checks your groups.
   If your user is in the file's group, the group permissions are used.
3. If you are not the owner and not in the group, the other permissions are used.

2. Example: a shared file

```
$ ls -l project.txt
-rw-r----- 1 alice developers 1024 Aug 10 10:05 project.txt
```

- Permissions (rw-r----)
- Owner (alice)
- Group (developers)
- File name (project.txt)

3. How different users get access



alice (file owner)



project.txt

Uses owner permissions (rw-)
- ✓ Can read
- ✓ Can write
- ✗ Cannot execute



bob (in developers group)



project.txt

Uses group permissions (r--)
- ✓ Can read
- ✗ Cannot write
- ✗ Cannot execute



carol (not in group)



project.txt

Uses others permissions (---)
- ✗ Cannot read
- ✗ Cannot write
- ✗ Cannot execute

4. Important: permissions are not added together

Common beginner mistake

Thinking that if a user gets permissions from multiple categories, Linux adds them together.

Example:
If bob is in the developers group, he does not get the owner's write permission.
He only gets the group permissions (r--).

Correct understanding

Linux uses one set of permissions based on your identity:

- If you are the owner → use owner permissions
- If you are in the group → use group permissions
- Otherwise → use others permissions

The categories are not combined. Only one set of permissions is used.

Engineering insight: A permission error is often a security boundary working correctly. It means Linux is doing its job by blocking an action that you are not allowed to perform.

Read, Write, Execute: What Can You Actually Do?

1. What each permission means (regular files)

Permission	Symbol	What it allows	Real example
Read	r	View the file content.	Read a script, config file, or text file.
Write	w	Modify the file content.	Edit a script, update a config file, or save changes.
Execute	x	Run the file as a program or script.	Execute a shell script or binary.

Key point

These permissions apply to regular files (not directories). Each permission controls a different action.

Important

A file can be readable and writable but not executable.

You need the execute (x) permission to run a script directly.

2. Example: The same script with different permissions

Read only (r--)

```
$ ls -l script.sh
-r--r--r-- 1 alice dev 120 script.sh
```



- ✓ Can read content
- ✗ Cannot modify
- ✗ Cannot execute

Read and write (rw-)

```
$ ls -l script.sh
-rw-r--r-- 1 alice dev 120 script.sh
```



- ✓ Can read content
- ✓ Can modify content
- ✗ Cannot execute

Read, write and execute (rwx)

```
$ ls -l script.sh
-rwxr-xr-x 1 alice dev 120 script.sh
```



- ✓ Can read content
- ✓ Can modify content
- ✓ Can execute (run)

3. Try it yourself

1. Create a simple script file.
   ```
   echo "Hello from VERIQTA" > script.sh
   ```
2. Check its permissions.
   ```
   ls -l script.sh
   ```
3. Add execute permission.
   ```
   chmod +x script.sh
   ```
4. Run the script.
   ```
   ./script.sh
   ```

4. What happens if you try to run without x?

```
$ ./script.sh
bash: ./script.sh: Permission denied
```

✗ You cannot run the script directly without execute permission.

Engineering insight

A readable script is not necessarily executable.

You can always run a readable script with an interpreter, such as: `bash script.sh`, even if it does not have the execute permission.

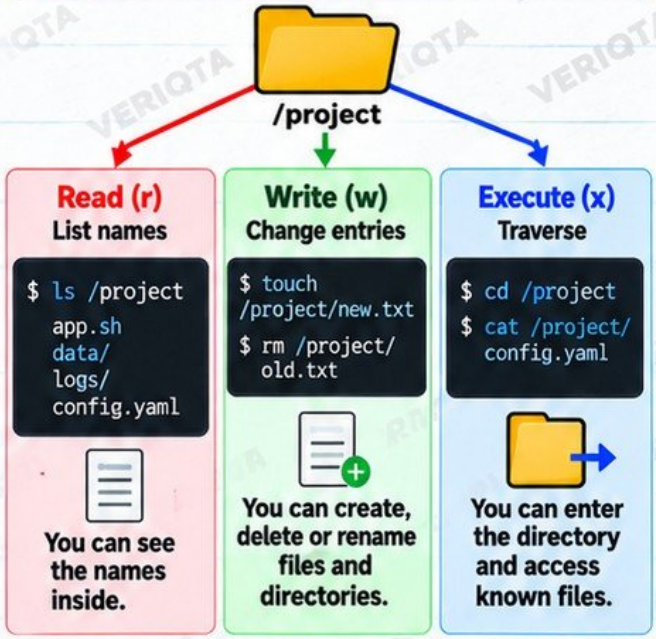
Directory Permissions Work Differently

1. What directory permissions mean

- Directory permissions control what you can do with the directory's contents, not the file content inside the files.

Permission	Symbol	What it allows on a directory	Real example
Read	r	List the names of files and subdirectories.	Run `ls` to see what's inside.
Write	w	Create, delete, and rename files and directories inside.	Create a new file, delete a file, or rename a directory.
Execute	x	Traverse or access the directory.	Enter the directory with `cd` or access a known file inside.

2. Visual: What each permission does



3. Different permission combinations

r-- (read only)

```
$ ls /project
file1.txt  app.sh
```

- ✓ Can list names
- ✗ Cannot create, delete or rename
- ✗ Cannot enter if execute is missing

--w (write only)

```
$ touch /project/test.txt
$ ls /project
```

- ✗ Cannot list names
- ✓ Can create, delete or rename files (when you know the name)
- ✗ Cannot enter if execute is missing

--x (execute only)

```
$ cd /project
$ ls
```

- ✗ Cannot list names
- ✗ Cannot create, delete or rename
- ✓ Can enter and access known files

rwx (full access)

```
$ ls /project
$ touch /project/test.txt
$ cd /project
```

- ✓ Can list names
- ✓ Can create, delete, rename
- ✓ Can enter and access files

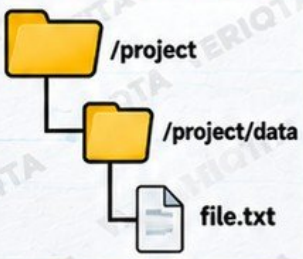
4. Common failure: No execute permission

You might have read or write permission, but if you do not have execute permission on the directory, you cannot enter it or access files inside.

```
$ cd /project
bash: cd: /project: Permission denied
```

- Even if you know a file exists, you cannot access it without execute (x) on the directory.
- Execute permission allows you to traverse the directory.

5. Real-world example: Parent directory matters



✗ If you do not have execute permission on /project, you cannot access /project/data/file.txt, even if you have permissions on the file.

Engineering insight: Directory permissions control your access to the directory itself and its entries. You need the right combination of r, w, and x depending on what you want to do.

Meet ls -l, stat, and id

Inspect permissions, ownership, identity, and group membership before making changes.

1. ls -l : View file permissions, owner, group, and details

```
$ ls -l app.sh
-rwxr-xr--  1 alice developers 1024 Aug 10 10:05 app.sh
```

- Permissions (rwxr-xr--)
- Owner (alice)
- Group (developers)
- File size, date and name

What you learn

- The first 10 characters show the file type and permissions.
- The next columns show the owner and group.
- You can quickly see who owns the file and what permissions are set.

2. stat filename : Detailed file information

```
$ stat app.sh
  File: app.sh
  Size: 1024        Blocks: 8         IO Block: 4096   regular file
Device: 8,1         Inode: 131072     Links: 1
Access: (0754/-rwxr-xr--)  Uid: ( 1000/alice)  Gid: ( 1001/developers)
Access: 2024-08-10 10:05:23.000000000 +0000
Modify: 2024-08-10 10:05:23.000000000 +0000
Change: 2024-08-10 10:05:23.000000000 +0000
 Birth: -
```

- Permissions (in both number and symbol)
- Owner (user ID and name)
- Group (group ID and name)

What you learn

- Shows detailed information about the file.
- Displays permissions in both symbolic and numeric form.
- Shows the owner (Uid) and group (Gid).
- Useful when you need more details than ls -l.

3. id : Show your current identity

```
$ id
uid=1000(alice) gid=1000(alice) groups=1000(alice),1001(developers)
```

- Your user ID and username
- Your primary group ID and group name
- All groups you belong to (including additional groups)

What you learn

- Shows your current user ID (uid).
- Shows your primary group ID (gid).
- Lists all groups you are a member of.
- Helps you understand which permissions may apply to you.

4. id -nG : Show only your group names

```
$ id -nG
alice developers docker sudo
```

- List of all group names (only names, easier to read)

What you learn

- Displays only the names of the groups you belong to.
- Useful for quickly checking if you are in a specific group, such as developers or sudo.

5. Practice: Identify ownership and permissions

1. Create a file and check its details.
   ```
   touch test.txt
   ls -l test.txt
   ```
2. Use stat to see more information.
   ```
   stat test.txt
   ```
3. Check your user identity and groups.
   ```
   id
   id -nG
   ```
4. Determine which permissions apply to you as the current user.

Engineering insight: Always inspect the current permissions, ownership, and your own identity before making changes. This helps you avoid unnecessary permission errors and make the right, safe changes.

Understand Permission Numbers: 644, 755, and 600

Read = 4, Write = 2, Execute = 1 . These values combine to give each permission.

1. Permission values

Permission	Symbol	Value	What it allows
Read	r	4	View the file content (or list directory names).
Write	w	2	Modify the file content (or create/delete files in a directory).
Execute	x	1	Run the file as a program (or enter/traverse a directory).
No permission	-	0	Not allowed.

2. How the values combine

Add the values for the permissions you want.

- r = 4
- w = 2
- x = 1

- Add the numbers for each category (owner, group, others).

Example:

rw- = 4 + 2 + 0 = 6

r-x = 4 + 0 + 1 = 5

r-- = 4 + 0 + 0 = 4

3. Common permission combinations

6 = rw- (read and write)

- Read (4) ✓
- Write (2) ✓
- Execute (0) ✗

4 + 2 + 0 = 6

rw-

5 = r-x (read and execute)

- Read (4) ✓
- Write (0) ✗
- Execute (1) ✓

4 + 0 + 1 = 5

r-x

4 = r-- (read only)

- Read (4) ✓
- Write (0) ✗
- Execute (0) ✗

4 + 0 + 0 = 4

r--

4. Real examples: 644, 755, and 600

644 - Common for regular files

```
$ ls -l file1.txt
-rw-r--r-- 1 alice dev 1024 Aug 10 10:05 file1.txt
```

Owner	Group	Others
rw-	r--	r--
(6)	(4)	(4)
Read & write	Read only	Read only

755 - Common for directories or scripts

```
$ ls -l script.sh
-rwxr-xr-x 1 alice dev 1024 Aug 10 10:05 script.sh
```

Owner	Group	Others
rwx	r-x	r-x
(7)	(5)	(5)
Read, write and execute	Read & execute	Read & execute

600 - Common for private files (e.g., SSH keys)

```
$ ls -l id_rsa
-rw------- 1 alice dev 1679 Aug 10 10:05 id_rsa
```

Owner	Group	Others
rw-	---	---
(6)	(0)	(0)
Read & write	No access	No access

5. Quick reference table

Number	Owner	Group	Others	Common use
600	rw-	---	---	Private files (e.g., SSH keys)
644	rw-	r--	r--	Regular files (e.g., configs, text files)
755	rwx	r-x	r-x	Directories or executable scripts
770	rwx	rw-	---	Shared team directories
777	rwx	rwx	rwx	Not recommended (world write access)

6. Practice: Convert between symbolic and numeric

1. Convert the following to numbers:
   - rw- → ?
   - r-x → ?
   - r-- → ?
2. What is the permission number for:
   a) rw-r--r-- b) rwxr-xr-x c) rw------
3. Create a file and set its permission to 644, then verify it.

```
touch demo.txt
chmod 644 demo.txt
ls -l demo.txt
```

7. Engineering insight

Permission numbers represent access combinations, not security scores.

- A higher number like 777 does not mean "more secure." It simply allows more access.

Change Permissions With chmod

1. Symbolic permission changes

Symbol	Meaning	Example	Result
u	user (owner)	chmod u+x file.txt	Add execute for owner
g	group	chmod g-w file.txt	Remove write for group
o	others	chmod o-r file.txt	Remove read for others
a	all (user, group, others)	chmod a+x file.txt	Add execute for everyone

2. Symbolic examples with before and after

Add execute for owner (u+x)

```
$ ls -l script.sh
-rw-r--r-- 1 alice dev 123 script.sh
```

→

```
$ chmod u+x script.sh
$ ls -l script.sh
-rwxr--r-- 1 alice dev 123 script.sh
```

Remove write for group (g-w)

```
$ ls -l file.txt
-rw-rw-r-- 1 alice dev 123 file.txt
```

→

```
$ chmod g-w file.txt
$ ls -l file.txt
-rw-r--r-- 1 alice dev 123 file.txt
```

Remove read for others (o-r)

```
$ ls -l notes.txt
-rw-r--r-- 1 alice dev 123 notes.txt
```

→

```
$ chmod o-r notes.txt
$ ls -l notes.txt
-rw-r----- 1 alice dev 123 notes.txt
```

3. Numeric permission changes

Number	Symbolic	What it means
7	rwx	Read, write and execute
6	rw-	Read and write
5	r-x	Read and execute
4	r--	Read only
3	-wx	Write and execute
2	-w-	Write only
1	--x	Execute only
0	---	No permission

4. Numeric example: chmod 640

```
$ ls -l config.ini
-rw-r--r-- 1 alice dev 123 config.ini
```

→

```
$ chmod 640 config.ini
$ ls -l config.ini
-rw-r----- 1 alice dev 123 config.ini
```

Before (rw-r--r--)
Owner: read+write
Group: read only
Others: read only

After (rw-r-----)
Owner: read+write
Group: read only
Others: no access

5. Combine multiple symbolic changes

```
$ ls -l data.sh
-rw-rw-r-- 1 alice dev 123 data.sh
$ chmod u+x,g-w,o-r data.sh
$ ls -l data.sh
-rwxr--r-- 1 alice dev 123 data.sh
```

What happened?

- u+x → added execute for owner
- g-w → removed write for group
- o-r → removed read for others

6. Use stat to verify permissions

```
$ stat data.sh
  File: data.sh
  Size: 123       Blocks: 8       IO Block: 4096   regular file
Device: 8,1       Inode: 131072   Links: 1
Access: (0754/-rwxr--r--)  Uid: ( 1000/alice)  Gid: ( 1001/dev)
Access: 2024-08-10 10:05:23.000000000 +0000
Modify: 2024-08-10 10:05:23.000000000 +0000
Change: 2024-08-10 10:05:23.000000000 +0000
```

7. Practice: Create a file and change its permissions

1. Create a test file.
   ```
   touch test.txt
   ```
2. Check its current permissions.
   ```
   ls -l test.txt
   stat test.txt
   ```
3. Change permissions using symbolic mode.
   ```
   chmod u+x test.txt
   chmod g-w test.txt
   ```
4. Change permissions using numeric mode.
   ```
   chmod 640 test.txt
   ls -l test.txt
   stat test.txt
   ```

Engineering insight

Permission numbers represent access combinations, not security scores.

A higher number like 777 simply allows more access. The right permissions depend on what each user needs to do.

Why chmod 777 Is Not a Real Fix

1. What chmod 777 permits

777 gives read, write and execute to everyone (owner, group, and others).

```
$ ls -l script.sh
-rwxrwxrwx 1 alice developers 123 Aug 10 10:05 script.sh
```

Owner	Group	Others
rwx	rwx	rwx
(7)	(7)	(7)
Read, write, execute	Read, write, execute	Read, write, execute

2. Why this is a problem

Anyone can modify the file.
Any user on the system can change the content, which can lead to accidental or malicious changes.

Increases the risk of malicious code.
Attackers can replace scripts or configuration files with harmful content.

Accidental deletion is possible.
Anyone can delete or rename the file or directory.

Breaks the principle of least privilege.
It gives more access than necessary, which makes the system less secure.

3. Real-world example: Overly permissive vs least privilege

Overly permissive (777)

```
$ ls -l app.sh
-rwxrwxrwx 1 alice dev 123 Aug 10 10:05 app.sh
```

- ✗ Any user can modify the script.
- ✗ Any user can delete the script.
- ✗ Anyone can execute it.
- ✗ High risk on multi-user systems and production servers.

Least privilege (750)

```
$ ls -l app.sh
-rwxr-x--- 1 alice dev 123 Aug 10 10:05 app.sh
```

- ✓ Owner can read, write and execute.
- ✓ Group can read and execute.
- ✓ Others have no access.
- ✓ Only the required users can use or modify the file.

4. Investigate the real requirement

Ask these questions before changing permissions:

1. Who needs access to this file or directory?
2. Do they need to read, write, or execute (or all three)?
3. Is a group more appropriate than giving access to everyone?
4. Can we achieve the requirement with smaller, specific permissions?
5. Is there a better solution, such as setting the correct owner or group, instead of using 777?

5. Example: Fixing the real problem

Instead of using 777, give only the access that is required.

```
# If a script only needs to be executed by the owner and group
$ chmod 750 deploy.sh
$ ls -l deploy.sh
-rwxr-x--- 1 alice dev 123 Aug 10 10:05 deploy.sh
```

- ✓ Owner can read, write, execute.
- ✓ Group can read, execute.
- ✓ Others cannot access.

6. Directories: Even more risky

chmod 777 on a directory allows anyone to create, delete, or modify files inside it.

```
$ ls -ld /var/app
drwxrwxrwx 3 alice dev 4096 Aug 10 10:05 /var/app
```

- ✗ Anyone can create files.
- ✗ Anyone can delete files.
- ✗ Anyone can replace files with malicious content.
- ✗ This can break your application or compromise your system.

7. Security reminder

Engineering insight
Permission numbers represent access combinations, not security scores.

Determine which identity needs which access, and set the smallest permissions that meet the requirement.

Change Ownership With chown and chgrp

1. Ownership vs Permissions

Ownership

- Tells you who owns the file (owner and group).
- Controlled by **chown** and **chgrp**.
- Decides which users and groups are considered for access.

Permissions

- Tells you what the owner, group, and others can do (read, write, or execute).
- Controlled by **chmod**.
- Works together with ownership to determine access.

2. Key Commands

Command	Purpose	Example
chown	Change the owner of a file or directory.	`chown alice app.conf`
chgrp	Change the group of a file or directory.	`chgrp developers app.conf`
Privileges required	You need root or sudo to change ownership.	`sudo chown alice:dev app.conf`

3. Visual: File changing from one owner and group to another

Before

```
$ ls -l app.conf
-rw-r--r-- 1 root root 1024 Aug 10 10:05 app.conf
```

Owner: root
Group: root

After

```
$ ls -l app.conf
-rw-r--r-- 1 alice developers 1024 Aug 10 10:05 app.conf
```

Owner: alice
Group: developers

4. How to change ownership

1. Change owner only:
   `sudo chown alice app.conf`
2. Change group only:
   `sudo chgrp developers app.conf`
3. Change both owner and group:
   `sudo chown alice:developers app.conf`

5. Real-world example: Preparing an application-owned directory

Initial state (wrong owner and group)

```
$ ls -ld /var/www/myapp
drwxr-xr-x 2 root root 4096 Aug 10 10:05 /var/www/myapp
```

- The directory is owned by root and group **root**.
- The application cannot write to it.
- Other users may have unnecessary access.

After fixing ownership

```
$ ls -ld /var/www/myapp
drwxrwx--- 2 appuser developers 4096 Aug 10 10:05 /var/www/myapp
```

- Now owned by the application user (appuser).
- Belongs to the developers group.
- Access is restricted to only the right people.

6. Common mistakes

- Changing ownership without proper privileges.
- Forgetting to set the correct group.
- Giving ownership to the wrong user or group.
- Using 777 instead of proper ownership and group settings.

7. Practice: Try it yourself

1. Create a test file:
   `touch app.conf`
2. Change the owner:
   `sudo chown alice app.conf`
3. Change the group:
   `sudo chgrp developers app.conf`
4. Verify the result:
   `stat app.conf`

Engineering insight

File ownership and permissions are different but work together.

Changing ownership is not about making the file "more secure," but about assigning the right control to the right user and group.

Security reminder: Always verify which identity needs access before changing ownership or permissions.

Change Ownership With chown and chgrp

1. Ownership vs Permissions

Ownership

- Shows who owns the file or directory.
- Controlled by user (owner) and group.
- Changed using chown and chgrp.

Permissions

- Control what actions can be performed (read, write, execute).
- Set using numbers or symbolic mode.
- Does not change the owner or group.

2. Key Commands

Command	Purpose	Example
chown	Change the owner of a file or directory.	`chown alice app.conf`
chgrp	Change the group of a file or directory.	`chgrp developers app.conf`
Both	Can be combined to change owner and group together.	`chown alice:developers app.conf`

Privileges required:
- You must be the file owner, or
- You must have sudo/root privileges.

3. Visual: Changing owner and group

Before

```
$ ls -l app.conf
-rw-r--r-- 1 root root 1024 Aug 10 10:05 app.conf
```

After

```
$ ls -l app.conf
-rw-r--r-- 1 alice developers 1024 Aug 10 10:05 app.conf
```

4. Using chown

Change the owner of a file

```
$ chown alice app.conf
$ ls -l app.conf
-rw-r--r-- 1 alice root 1024 Aug10 10:05 app.conf
```

- Changes only the owner.
- Requires the right privileges.

5. Using chgrp

Change the group of a file

```
$ chgrp developers app.conf
$ ls -l app.conf
-rw-r--r-- 1 root developers 1024 Aug 10 10:05 app.conf
```

- Changes only the group.
- Requires the right privileges.

6. Using both (recommended)

Change owner and group together

```
$ chown alice:developers app.conf
$ ls -l app.conf
-rw-r--r-- 1 alice developers 1024 Aug 10 10:05 app.conf
```

- Changes owner and group in one command.
- Useful for application deployments.

Real-world example: Preparing an application-owned directory

1. Create the application directory:
   `mkdir -p /var/www/myapp`
2. Assign the correct owner and group:
   `chown www-data:www-data /var/www/myapp`
3. Set appropriate permissions:
   `chmod 750 /var/www/myapp`

```
$ ls -ld /var/www/myapp
drwxr-x--- 2 www-data www-data 4096 Aug 10 10:10 /var/www/myapp
```

- Application can access what it needs.
- Other users cannot modify it.
- Keeps the system secure.

Key takeaways

- Ownership and permissions are different.
- Use chown to change the owner.
- Use chgrp to change the group.
- Use sudo or be the owner to make these changes.
- Always set the minimum access required.

Engineering insight: Changing ownership is not about giving more access, but about giving the right access to the right identity. Always verify the real requirement before making changes.

Troubleshoot Permission denied

1. The common error

```
$ cat /opt/app/config.yaml
cat: /opt/app/config.yaml:
Permission denied
```

You might see "Permission denied" when you try to read, write, execute a file, or access a directory.

2. Common reasons

- Insufficient permissions (r, w, or x missing).
- Incorrect ownership (wrong user or group).
- Inaccessible parent directory (missing x on a directory in the path).
- Trying to execute a file without execute permission.
- Using the wrong user or not being in the required group.

3. Diagnostic commands

Command	What it shows
id	Your current user and groups.
ls -ld <path>	Permissions, owner and group of a file or directory.
namei -l <path>	Permissions of each directory in the path.
stat <path>	Detailed ownership, permissions and metadata.

4. Step-by-step troubleshooting flow

1 Reproduce the error

```
$ cat /opt/app/config.yaml
cat: /opt/app/config.yaml: Permission denied
```

2 Check your identity

```
$ id
uid=1001(alice) gid=1001(alice)
groups=1001(alice),1002(developers)
```

3 Check the file or directory permissions

```
$ ls -l /opt/app/config.yaml
-rw------- 1 root root 4096 Aug 10 10:05 /opt/app/config.yaml
```

4 Check the parent directories

```
$ namei -l /opt/app/config.yaml
f: /opt/app/config.yaml
 drwxr-xr-x     root        root      /
 drwxr-xr-x     opt         root      opt
 drwx------     app         root      app
 -rw*------     config.yaml root      root   config.yaml
```

5 Understand the issue

The file is owned by **root** and only root has read access.
Your user (alice) does not have permission.

6 Apply the correct fix

```
$ sudo chown alice:developers /opt/app/config.yaml
$ chmod 640 /opt/app/config.yaml
```

7 Verify the result

```
$ ls -l /opt/app/config.yaml
-rw-r----- 1 alice developers 4096 Aug 10 10:05 /opt/app/config.yaml
$ stat /opt/app/config.yaml | grep "Access:"
Access: (0640/-rw-r-----)  Uid: ( 1001/alice)   Gid: ( 1002/developers)
```

5. More real-world examples

A. No execute permission on a directory

```
$ cd /opt/app
bash: cd: /opt/app: Permission denied
```

Check and fix:

```
$ ls -ld /opt/app
drw------ 2 root root 4096 Aug 10 10:05 /opt/app
$ sudo chmod 750 /opt/app
```

You need execute permission (x) on a directory to enter or access files inside it, even if you have read or write permissions.

B. Write permission missing

```
$ echo "test" >> /var/log/app.log
bash: /var/log/app.log: Permission denied
```

Check and fix:

```
$ ls -l /var/log/app.log
-rw-r--r-- 1 root root 1024 Aug 10 10:05 /var/log/app.log
$ sudo chown alice:developers /var/log/app.log
$ chmod 664 /var/log/app.log
```

You need write permission (w) to modify a file. Either change the ownership or give the required write permission.

C. File is readable but not executable

```
$ ./deploy.sh
bash: ./deploy.sh: Permission denied
```

Check and fix:

```
$ ls -l deploy.sh
-rw-r--r-- 1 alice developers 123 Aug 10 10:05 deploy.sh
$ chmod +x deploy.sh
$ ls -l deploy.sh
-rwxr-xr-x 1 alice developers 123 Aug 10 10:05 deploy.sh
```

You need execute permission (x) to run a file or script.

6. Key takeaways

- Always check who you are (id).
- Check permissions, owner and group (ls -ld, stat).
- Verify the entire path (namei -l).
- Determine the exact access needed (read, write, or execute).
- Fix the root cause instead of using 777.

Why a File Can Be Read but Not Executed

1. The key difference

A file can be readable (r) but not executable (x).
You can still run a script by passing it to an interpreter like bash, even if it does not have execute permission.

2. Our test script

```
$ cat script.sh
#!/bin/bash
echo "Hello from VERIQTA!"
```

This is a simple script.

- It has a shebang (#!/bin/bash).
- It prints a message.
- We will test two ways to run it.

3. Run directly with ./script.sh (requires execute permission)

File without execute permission (644)

```
$ ls -l script.sh
-rw-r--r-- 1 alice developers 42 Aug 10 10:05 script.sh
```

```
$ ./script.sh
bash: ./script.sh: Permission denied
```

- The system checks the execute (x) permission before running the file directly.
- Since the file is not executable, it is denied.

4. Run with an interpreter: bash script.sh

Same file without execute permission (644)

```
$ ls -l script.sh
-rw-r--r-- 1 alice developers 42 Aug 10 10:05 script.sh
```

```
$ bash script.sh
Hello from VERIQTA!
```

- The bash command reads the file content.
- Only read permission (r) is required.
- The script is executed by the bash interpreter, not by the file itself.

5. After adding execute permission

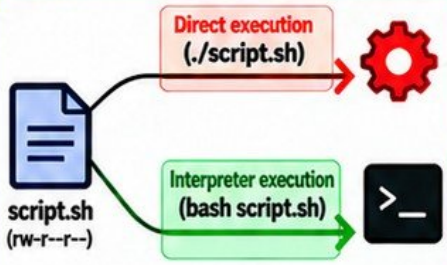
```
$ chmod +x script.sh
$ ls -l script.sh
-rwxr-xr-x 1 alice developers 42 Aug 10 10:10 script.sh
```

Now you can run it directly

```
$ ./script.sh
Hello from VERIQTA!
```

After adding execute permission (x), you can run the script directly with ./script.sh.

6. Visual: Two execution paths



System checks:

- Is the file executable (x)?
- If not, Permission denied.
- Requires execute permission.

bash reads the file:

- Only read permission (r) is needed.
- The file is passed as input to bash.
- Works even without execute permission.

7. Practice: Try it yourself

1. Create a test script.
```
echo '#!/bin/bash' > test.sh
echo 'echo "Hello from VERIQTA!"' >> test.sh
```
2. Check current permissions.
```
ls -l test.sh
```
3. Try running it directly.
```
./test.sh    # Expected: Permission denied
```
4. Run it with bash.
```
bash test.sh    # Expected: Works
```
5. Add execute permission and run again.
```
chmod +x test.sh
./test.sh       # Expected: Works
```

8. Key takeaways

- Read permission (r) lets you view the file content.
- Execute permission (x) lets you run the file directly.
- An interpreter (like bash) can run a readable script without execute permission.
- Always use the method that matches your need and security requirements.

Engineering insight

- A readable script is not necessarily executable.
- Execution permission controls whether the file can be run directly, but interpreters only need to read it.
- Be intentional with permissions and give execute access only when it is required.

Why You Cannot Delete a File You Own

1. The key concept

Deleting a file depends on permissions on its parent directory, not the file's own write bit.

2. What happens when you delete a file?

- A file is a name (directory entry) that points to data on disk.
- Deleting a file removes the name from the directory.
- This is why you need write permission (w) on the directory to delete a file, even if you own the file.

3. Example: You own the file, but cannot delete it

Check the directory permissions (no write)

```
$ ls -ld /restricted
dr-xr-xr-x 2 root root 4096 Aug 10 10:05 /restricted
```

You can read and enter the directory (r-x), but you do NOT have write permission (w).

Check the file permissions (you own it)

```
$ ls -l /restricted/myfile.txt
-rw-r-r-x 1 alice alice 123 Aug 10 10:05 /restricted/myfile.txt
```

You own the file and it is writable.

Try to delete the file

```
$ rm /restricted/myfile.txt
rm: cannot remove '/restricted/myfile.txt':
Permission denied
```

- You own the file.
- The file is writable.
- But you cannot delete it because you do not have write permission on the parent directory.

4. Fix: Add write permission to the directory

Check current directory permissions

```
$ ls -ld /restricted
dr-xr-xr-x 2 root root 4096 Aug 10 10:05 /restricted
```

Add write permission for the owner (or group)

```
$ sudo chmod u+w /restricted
```

Verify the new permissions

```
$ ls -ld /restricted
drwxr-xr-x 2 root root 4096 Aug 10 10:05 /restricted
```

Now you can delete the file

```
$ rm /restricted/myfile.txt
$ ls /restricted
```

After the directory has write permission, you can delete the file.

5. Sticky bit restrictions (common example)

```
$ ls -ld /tmp
drwxrwxrwt 10 root root 4096 Aug 10 10:05 /tmp
```

- The sticky bit (t) on a directory (like /tmp) allows everyone to create files, but only the file owner, directory owner (root), or a privileged user can delete or rename files.
- This prevents users from deleting each other's files.

6. Other controls that can prevent deletion

- No write permission on the parent directory.
- Sticky bit restrictions (as shown with /tmp).
- Immutable attribute (chattr +i) for special protection.
- File system mounted as read-only.
- Access control lists (ACLs) with restrictive rules.

7. Key takeaways

- Deleting a file requires write permission on its parent directory.
- The file's own write bit does not control deletion.
- Look at directory permissions, ownership, sticky bit, and other controls.
- Always investigate the actual cause instead of assuming a permission problem with the file.

Engineering insight

- Deleting a file changes a directory entry, not the file's content.
- Directory permissions control who can add, remove, or rename files in that directory.
- Sticky bit and other controls add extra protection, especially on shared directories.

The Hidden Influence of umask

1. What is umask?

- umask is a process setting that removes certain permissions from new files and directories.
- It does not change existing files.
- It acts as a filter to limit the default permissions when you create something.

2. Check your current umask

```
$ umask
022
$ umask -S
u=rwx,g=rx,o=rx
```

The output shows which permissions are removed for new files and directories.

3. Default permissions before umask

Linux starts with default permissions and then subtracts the umask.

Type	Default	Meaning
File	666	rw-rw-rw- (read and write for everyone)
Directory	777	rwxrwxrwx (read, write, execute for everyone)

4. How umask changes the permissions

The umask value is subtracted from the default permissions.

Item	Default	umask	Result	Meaning
File	666	022	644	rw-r--r-- (read/write for owner, read for group and others)
Directory	777	022	755	rwxr-xr-x (read, write, execute for owner, read and execute for group and others)

5. Real example: Create a file and directory with umask 022

```
$ umask
022
$ touch testfile.txt
$ mkdir testdir
$ ls -ld testfile.txt testdir
-rw-r--r-- 1 alice developers 0 Aug 10 10:05 testfile.txt
drwxr-xr-x 2 alice developers 4096 Aug 10 10:05 testdir
```

- The file is created with 644 permissions.
- The directory is created with 755 permissions.

6. Change the umask and compare

umask 077 (more restrictive)

```
$ umask 077
$ touch private.txt
$ mkdir private_dir
$ ls -ld private.txt private_dir
-rw------- 1 alice developers 0 Aug 10 10:05 private.txt
drwx------ 2 alice developers 4096 Aug 10 10:05 private_dir
```

- Files are created with 600.
- Directories are created with 700.
- Only the owner has access.

umask 002 (more open for group)

```
$ umask 002
$ touch shared.txt
$ mkdir shared_dir
$ ls -ld shared.txt shared_dir
-rw-rw-r-- 1 alice developers 0 Aug 10 10:05 shared.txt
drwxrwxr-x 2 alice developers 4096 Aug 10 10:05 shared_dir
```

- Files are created with 664.
- Directories are created with 775.
- Group members also get write access.

7. Useful commands

Command	Purpose
umask	Show current umask (in numeric form).
umask -S	Show current umask (in symbolic form).
touch <file>	Create a new file.
mkdir <dir>	Create a new directory.
stat <file>	Show detailed file information and permissions.

8. Common beginner mistakes

- ✗ Assuming new files should be executable.
- ✗ Expecting new files to have 777 permissions.
- ✗ Forgetting that umask only affects newly created files and directories.
- ✗ Not understanding why directories get execute permission by default.

9. Key takeaways

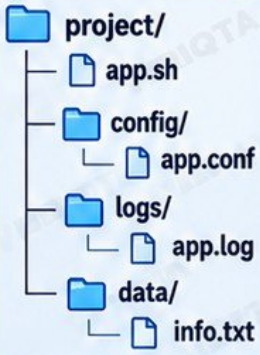
- ✓ umask removes permissions from the default values.
- ✓ Typical umask 022 creates files with 644 and directories with 755.
- ✓ Use a more restrictive umask (like 077) when you need private files.
- ✓ umask does not change existing files.
- ✓ New files do not need to be executable unless they are scripts or programs.

Recursive Permissions: The Danger of chmod -R

1. What does chmod -R do?

- chmod -R (recursive) applies the specified permissions to a directory and all of its files and subdirectories.
- It changes permissions on everything below the target path.
- It does not distinguish between files and directories, so the same permissions are applied to both.

2. Example directory structure (before changes)




```
$ ls -lR project
project:
total 4
-rwxr-xr-x 1 alice developers   52 app.sh
drwxr-xr-x 2 alice developers 4096 config
drwxr-xr-x 2 alice developers 4096 logs
drwxr-xr-x 2 alice developers 4096 data
project/config:
-rw-r--r-- 1 alice developers  120 app.conf
project/logs:
-rw-r--r-- 1 alice developers    0 app.log
project/data:
-rw-r--r-- 1 alice developers   30 info.txt
```

3. Apply a recursive permission change

```
$ chmod -R 644 project
$ ls -lR project
```

Now both files and directories have the same permissions.

```
project:
-rw-r--r-- 1 alice developers   52 app.sh
-rw-r--r-- 1 alice developers  120 app.conf
-rw-r--r-- 1 alice developers    0 app.log
-rw-r--r-- 1 alice developers   30 info.txt
-rw-r--r-- 2 alice developers 4096 config
-rw-r--r-- 2 alice developers 4096 logs
-rw-r--r-- 2 alice developers 4096 data
```

- Directories no longer have execute (x) permission.
- You cannot enter the directories.
- Scripts and programs are no longer executable.
- Applications may stop working.

4. What can go wrong?

Directories become inaccessible
Without execute (x) on directories, you cannot enter them even if you have read permission.

Scripts stop running
If scripts lose execute permission, they cannot be run directly.

Security risks
Using chmod -R 777 gives everyone full access, which can allow unwanted modification or deletion of files.

Application issues
Web servers, databases, and other applications may fail if they lose the expected permissions.

5. A safer approach

```
# 1. Inspect first
$ ls -lR project
# 2. Set directory permissions (example: 755)
$ find project -type d -exec chmod 755 {} \;
# 3. Set file permissions (example: 644)
$ find project -type f -exec chmod 644 {} \;
```

- Inspect the directory structure and current permissions first.
- Use different permissions for directories and files.
- Only change what is necessary.
- Avoid using chmod -R 777.

6. After applying the safer approach

```
$ ls -lR project
project:
-rwxr-xr-x  1 alice developers   52 app.sh
drwxr-xr-x  2 alice developers 4096 config
drwxr-xr-x  2 alice developers 4096 logs
drwxr-xr-x  2 alice developers 4096 data
project/config:
-rw-r--r--  1 alice developers  120 app.conf
project/logs:
-rw-r--r--  1 alice developers    0 app.log
project/data:
-rw-r--r--  1 alice developers   30 info.txt
```

7. Key takeaways

- chmod -R changes permissions on all files and directories recursively.
- Using the same permissions for both can cause security or execution problems.
- Always inspect the target first.
- Use appropriate permissions for directories (usually 755) and files (usually 644).
- Only change what is required.

8. Engineering insight

- Recursive changes are powerful but dangerous.
- Understand the impact on both files and directories before running the command.
- Use the principle of least privilege.
- Check application requirements and existing permissions to avoid breaking functionality.

Secure SSH Private Keys

1. Why permissions matter

- An SSH private key gives full access to your servers.
- If the key is readable by other users, it can be copied and used without your knowledge.
- SSH checks the permissions and may reject a key if it is accessible to others.
- Always keep your private key private and protected.

2. Recommended permissions

Set restrictive permissions so only you can read and write the key.

```
$ chmod 600 ~/.ssh/id_ed25519
$ ls -l ~/.ssh/id_ed25519
-rw------- 1 alice alice 411 Aug 10 10:05
~/.ssh/id_ed25519
```

600 means:

- Read and write for the owner only.
- No permissions for group.
- No permissions for others.

3. Check the current permissions

Use stat to see detailed information about the key file.

```
$ stat ~/.ssh/id_ed25519
  File: /home/alice/.ssh/id_ed25519
  Size: 411        Blocks: 8
Access: (0600/-rw-------)
  Uid: (1001/alice)  Gid: (1001/alice)
Access: 2026-08-10 10:05:12
Modify: 2026-08-10 10:05:12
Change: 2026-08-10 10:05:12
```

4. Example: Overly accessible key (insecure)

```
$ ls -l ~/.ssh/id_ed25519
-rw-r--r-- 1 alice alice 411 Aug 10 10:05
~/.ssh/id_ed25519
```

- This key is readable by other users.
- SSH may reject this key.
- If not rejected, another user could copy and use it.

5. SSH may reject keys with open permissions

```
$ ssh -i ~/.ssh/id_ed25519 user@server
@@@
```

Shared Team Directories: The Setgid Bit

1. What is the setgid bit?

- The setgid bit on a directory makes new files and directories inherit the directory's group instead of the creator's primary group.
- This helps teams collaborate in a shared directory.
- The setgid bit does not give extra read, write, or execute permissions. It only controls group inheritance for new items.

2. Create a shared team directory

```
$ sudo groupadd devteam
$ sudo mkdir -p /project
$ sudo chgrp devteam /project
$ sudo chmod 2775 /project
```

The 2 at the beginning sets the setgid bit on the directory. The rest (775) gives read, write, and execute permissions for the owner and group, and read and execute for others.

3. Check the setgid directory

```
$ ls -ld /project
drwxrwsr-x 2 root devteam 4096 Aug 10 10:05 /project
```

The s in the group execute position shows that the setgid bit is set on this directory.

4. Two engineers creating files in the shared directory

Engineer 1 (alice) creates a file

```
$ sudo -u alice touch /project/file1.txt
$ ls -l /project
-rw-rw-r-- 1 alice devteam 0 Aug 10 10:10 file1.txt
```



File is created by alice, but the group is devteam because of the setgid bit.

Engineer 2 (bob) creates a file

```
$ sudo -u bob touch /project/file2.txt
$ ls -l /project
-rw-rw-r-- 1 bob devteam 0 Aug 10 10:12 file2.txt
```



File is created by bob, but the group is still devteam.

5. Create a subdirectory

```
$ sudo -u alice mkdir /project/docs
$ ls -ld /project/docs
drwxrwsr-x 2 alice devteam 4096 Aug 10 10:15 /project/docs
```

New directories also inherit the devteam group and keep the setgid bit set (s).

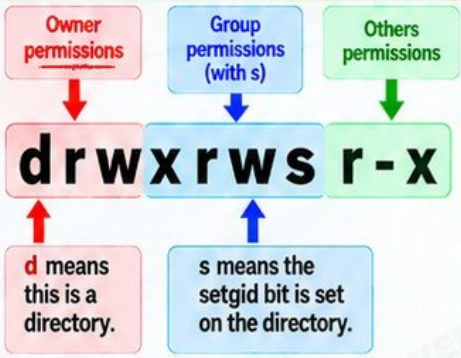
6. How to set the setgid bit

```
$ chmod g+s /project
# or use the numeric form:
$ chmod 2775 /project
$ ls -ld /project
drwxrwsr-x 2 root devteam 4096 Aug 10 10:05 /project
```

You can also remove the setgid bit with:

```
$ chmod g-s /project
```

7. Understanding the permission string



8. Practice: Inspect and verify

1. Check the permissions of the shared directory.
   ```
   ls -ld /project
   ```
2. Identify the s in the permission string.
3. Create your own file in the directory and verify the group.
   ```
   touch /project/test.txt
   ls -l /project/test.txt
   ```
4. Create a subdirectory and check its permissions.
   ```
   mkdir /project/newdir
   ls -ld /project/newdir
   ```

9. Key takeaways

- The setgid bit helps team members collaborate in a shared directory.
- New files and directories inherit the directory's group.
- Look for s in the group execute position to confirm the setgid bit.
- The setgid bit only affects group inheritance, not read, write, or execute permissions.
- Use setgid with appropriate permissions (usually 2775) on team directories.

Engineering insight

- Use a shared group for team collaboration.
- Combine the setgid bit with proper group permissions.
- The setgid bit is commonly used for project directories, shared code repositories, and team workspaces.
- It helps keep files accessible to the team without changing the owner.

Sticky Bit and Safe Shared Temporary Directories

1. What is the sticky bit?

- The sticky bit on a directory prevents users from deleting or renaming files that they do not own.
- It allows everyone to create files in the directory, but only the file owner, the directory owner, or root can delete or rename a file.
- This is commonly used on shared temporary directories such as /tmp.

2. Check the permissions of /tmp

```
$ ls -ld /tmp
drwxrwxrwt 23 root root 4096 Aug 10 10:05 /tmp
```

Directory permissions (777) | Sticky bit (t) | Owned by root and group root

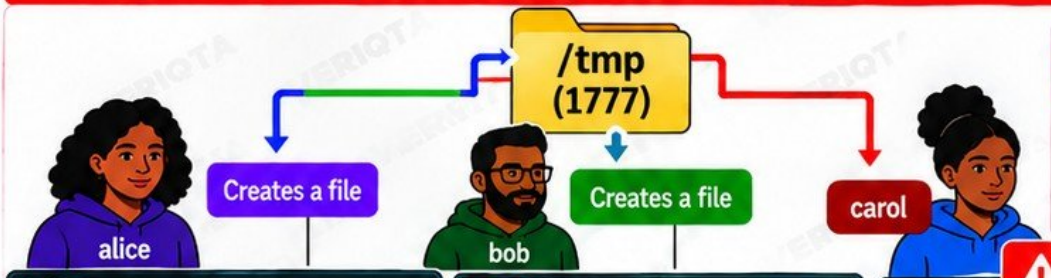
The t at the end shows that the sticky bit is set on this directory.

3. Numeric representation

1777 = 1000 111 111

1	Sticky bit (4th digit)
7	Owner permissions (rwx)
7	Group permissions (rwx)
7	Others permissions (rwx)

4. Multiple users working in a shared temporary directory (/tmp)



Even though /tmp is writable by everyone, carol cannot delete alice's file because the sticky bit restricts deletion to the file owner, directory owner, or root.

```
$ touch /tmp/alice.txt
$ ls -l /tmp/alice.txt
-rw-r--r-- 1 alice root 0 Aug 10 10:10 /tmp/alice.txt
```

```
$ touch /tmp/bob.txt
$ ls -l /tmp/bob.txt
-rw-r--r-- 1 bob root 0 Aug 10 10:12 /tmp/bob.txt
```

```
$ rm /tmp/alice.txt
rm: cannot remove '/tmp/alice.txt': Operation not permitted
```

5. Without the sticky bit (insecure)

```
$ sudo chmod 0777 /tmp/testdir
$ ls -ld /tmp/testdir
drwxrwxrwx 2 root root 4096 Aug 10 10:15 /tmp/testdir
$ touch /tmp/testdir/alice.txt
$ rm /tmp/testdir/alice.txt
# This succeeds. Any user can delete any file!
```

- Without the sticky bit, any user can delete or rename files created by other users.
- This can lead to accidental or malicious deletion of files.
- It is not safe for shared temporary directories.

6. With the sticky bit (secure)

```
$ sudo chmod 1777 /tmp/testdir
$ ls -ld /tmp/testdir
drwxrwxrwt 2 root root 4096 Aug 10 10:20 /tmp/testdir
$ touch /tmp/testdir/alice.txt
$ rm /tmp/testdir/alice.txt
rm: cannot remove '/tmp/testdir/alice.txt': Operation not permitted
```

- With the sticky bit, users can create their own files, but they cannot delete or rename files owned by other users.
- Only the file owner, directory owner, or root can delete or rename the file.
- This keeps shared temporary directories safe.

7. Other common examples

Directory	Permissions	Purpose
/tmp	1777	Shared temporary files (world writable with sticky bit)
/var/tmp	1777	Persistent temporary files (sticky bit)
/home	0755	User home directories (not world writable)

8. When to use 1777

- Use 1777 for shared temporary directories like /tmp and /var/tmp.
- It allows everyone to create files while preventing users from deleting each other's files.
- Do not use 1777 for regular project directories or application data.

9. Key takeaways

- The sticky bit prevents unauthorized deletion and renaming in shared directories.
- /tmp commonly uses 1777 for safe temporary file management.
- Users can create their own files but cannot delete others' files.
- Only the file owner, directory owner, or root can delete or rename a file.
- Use 1777 only for specific shared temporary-directory use cases, not as a universal setting.

Access Control Lists: When rwx Is Not Enough

1. What are Access Control Lists (ACLs)?

- ACLs allow you to grant specific permissions to named users or groups beyond the traditional owner, group, and others model.
- They provide more flexible and fine-grained access control.
- Useful when you need to give access to a specific user or group without changing the file's owner or primary group.

2. Default permissions (before ACL)

```
$ ls -l project.txt
-rw-r----- 1 alice devteam
1234 Aug 10 10:05 project.txt
```

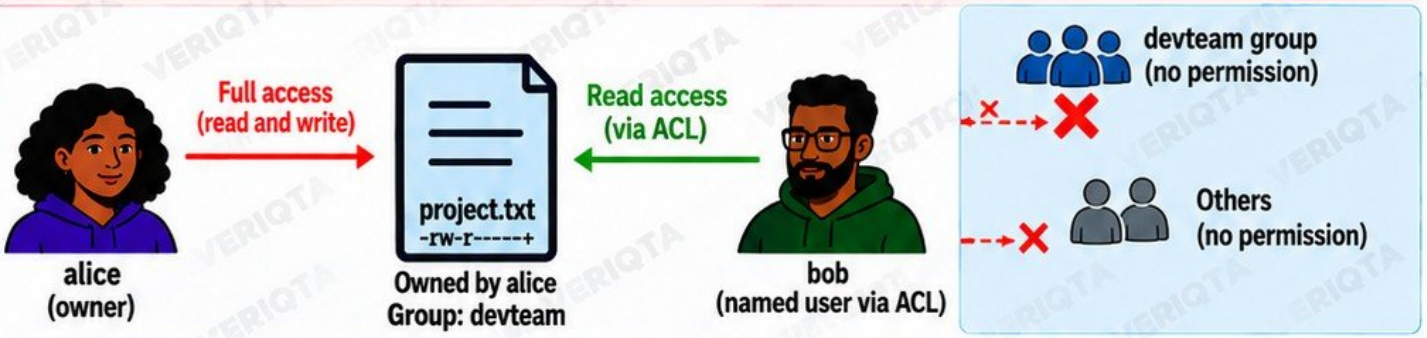
Only the owner (alice) can read and write. Group (devteam) and others have no access.

3. Grant a named user read access

```
$ setfacl -m u:bob:r project.txt
$ ls -l project.txt
-rw-r-----+ 1 alice devteam 1234
Aug 10 10:05 project.txt
```

We give user bob read access without changing the owner or group.

4. Visual: Granting access to a named engineer



5. View ACLs with getfacl

```
$ getfacl project.txt
# file: project.txt
# owner: alice
# group: devteam
user::rw-
user:bob:r--
group::---
mask::r---
other::---
```

- `user::rw-` — Standard owner permissions.
- `user:bob:r--` — Named user (bob) with read access.
- `mask::r---` / `other::---` — Group and others still have no access.

6. Important commands

Command	Purpose
`getfacl <file>`	Show ACL entries for a file or directory.
`setfacl -m u:<user>:<perms> <file>`	Grant permissions to a named user.
`setfacl -m g:<group>:<perms> <file>`	Grant permissions to a named group.
`setfacl -x u:<user> <file>`	Remove a named user's ACL entry.
`setfacl -b <file>`	Remove all ACL entries (keep only standard permissions).

7. The + indicator in ls -l

```
$ ls -l project.txt
-rw-r-----+ 1 alice devteam
1234 Aug 10 10:05 project.txt
```

The + at the end of the permission string indicates that the file has extended ACL entries.

8. Remove an ACL entry

```
$ setfacl -x u:bob project.txt
$ getfacl project.txt
# file: project.txt
# owner: alice
# group: devteam
user::rw-
group::---
other::---
```

9. Key takeaways

- ACLs let you grant access to specific users or groups beyond owner, group, and others.
- Use getfacl to view ACL entries.
- The + in ls -l indicates extended ACLs are present.
- ACLs are useful for team collaboration and special access requirements.
- Remove ACL entries when they are no longer needed.

10. Troubleshooting tips

- If a user still cannot access a file, check both ACLs and standard permissions.
- Use getfacl to verify the effective permissions.
- Remember that directory permissions also affect access (e.g., read and execute on the directory).
- Use setfacl -b to remove all ACL entries if needed.

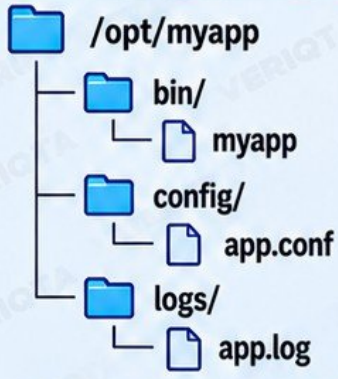
Production Challenge: Fix a Broken Application Directory

1. Scenario

After a deployment, the application cannot read its configuration file and cannot write to its log directory.

- The service fails to start or shows permission errors.
- The configuration file exists but cannot be read by the service.
- The log directory exists but the service cannot write logs.
- Your task is to investigate the problem, apply least-privilege permissions, and verify that the service works without making everything world-writable.

2. Application directory layout



The application runs as the user myapp and needs to:

- Read /opt/myapp/config/app.conf
- Write to /opt/myapp/logs/

3. Check the service and runtime user

```
$ systemctl status myapp
$ systemctl cat myapp | grep User
$ id myapp
uid=997(myapp) gid=995(myapp)
groups=995(myapp)
```

Identify the user and group the service runs as (myapp in this example).

4. Investigate the current permissions

```
$ ls -ld /opt/myapp
$ ls -l /opt/myapp/config/app.conf
$ ls -ld /opt/myapp/logs
$ namei -l /opt/myapp/config/app.conf
```

5. Example of the problem (before fixes)

```
$ ls -l /opt/myapp/config/app.conf
-rw------- 1 root root 600 Aug 10 10:05 app.conf
$ ls -ld /opt/myapp/logs
drwxr-xr-x 2 root root 755 Aug 10 10:05 logs
```

Service errors:

Cannot read configuration file: Permission denied

Cannot write to log directory: Permission denied

6. Apply least-privilege fixes

```
# Give read access to the config file
$ chown root:myapp /opt/myapp/config/app.conf
$ chmod 640 /opt/myapp/config/app.conf

# Allow the service to write logs
$ chown -R myapp:myapp /opt/myapp/logs
$ chmod 750 /opt/myapp/logs
```

- Configuration file: readable by root and myapp group, not by others.
- Log directory: owned by myapp and writable by the service, not world-writable.
- No unnecessary permissions are granted.

7. Verify the fixes

```
$ ls -l /opt/myapp/config/app.conf
-rw-r----- 1 root myapp 640 Aug 10 10:05 app.conf
$ ls -ld /opt/myapp/logs
drwxr-x--- 2 myapp myapp 750 Aug 10 10:05 logs
$ sudo -u myapp test -r /opt/myapp/config/app.conf
$ sudo -u myapp touch /opt/myapp/logs/test.log
$ ls -l /opt/myapp/logs/test.log
-rw-r--r-- 1 myapp myapp 0 Aug 10 10:20 test.log
```

The service user can read the configuration file and write to the log directory. The application works without using world-writable permissions.

8. Troubleshooting checklist

- Check the service's runtime user and group.
- Verify ownership and permissions on all components and parent directories.
- Use namei -l to confirm you can traverse each directory.
- Ensure the configuration file is readable by the service.
- Ensure the log directory is writable by the service.
- Apply the least permissions required.
- Verify the service reads the config and writes logs.

9. Commands used

```
id
ls -ld
ls -l
namei -l
chown
chmod
test
systemctl
```

10. Key takeaways

- Always check the service's runtime identity.
- File and directory permissions both matter.
- You need execute (x) on directories to traverse them.
- Give the service only the permissions it needs.
- Avoid world-writable permissions.
- Verify after making changes.

Engineering insight

- Application failures after deployment are often caused by incorrect ownership or permissions.
- Fixing permissions with least privilege improves security.
- Use appropriate groups and permissions instead of 777.